

基于RESNET的气象预测

1.数据下载

引入climate_learn库，并调用ClimateLearn核心的下载函数

```
cl.data.download_weatherbench(  
    dst=temperature_dst,  
    dataset="era5",  
    variable="temperature_850"  
)  
  
cl.data.download_weatherbench(  
    dst=geopotential_dst,  
    dataset="era5",  
    variable="geopotential_500"  
)
```

完整代码可见“1_download_data.py”

2.数据处理（从.nc到.npz）

由于从ERA5下载下来的数据是 .nc (NetCDF) 格式的。这是一种在气象和海洋学中非常流行的科学数据格式，它可以存储多维度的数据（比如经度、纬度、高度、时间）。

但是，对于深度学习模型训练来说，每次都从庞大的 .nc 文件里读取数据，效率很低。模型训练需要反复、快速地读取大量数据批次，我们需要一种更“亲近”模型的格式。

2.1 格式转化

将 .nc 文件转换成 .npz (NumPy 压缩数组) 格式。PyTorch 可以极其高效地读取NumPy数组，这能大大加快后续的训练速度。

2.2 数据切分

将从1979年到2018年这几十年的连续数据，按照标准的研究范式，切分成三个独立的集合：

- **训练集 (Training Set):** 用于“教”模型学习天气规律。我们使用 1979–2014 年的数据。
- **验证集 (Validation Set):** 用于在训练过程中“考核”模型，看看模型有没有学好，并用来调整模型的超参数（比如学习率）。我们使用 2015–2016 年的数据。
- **测试集 (Test Set):** 完全保留不用的“最终考卷”。只有当模型训练和验证全部完成后，才用它来评估模型的最终性能，得出最客观的评分。我们使用 2017–2018 年的数据。

2.3数据分片 (Sharding)

不要把所有训练数据都塞进一个巨大的文件里，而是把它切成16个小“分片”。这样做可以有效管理内存，训练时不需要一次性把几十GB的数据全加载进来。

```
convert_nc2npz(  
    root_dir=root_dir,  
    save_dir=save_dir,  
    variables=["temperature", "geopotential"], # 指定  
    要处理的变量  
    start_train_year=1979,  
    start_val_year=2015,  
    start_test_year=2017,  
    end_year=2018,  
    num_shards=16 # 将数据切成16个分片  
)
```

```
2_process_data x
D:\Anaconda\envs\rag_env\python.exe E:\Project_Collection\2025\WEATHER\CLIMATE_LEARN_PROJECT\2_process_data.py
ClimateLearn快速入门 - 阶段2: 处理数据
=====
本脚本将执行以下操作:
1. 读取 'data' 文件夹中的 .nc 文件。
2. 将数据切分为: 训练集 (1979-2014), 验证集 (2015-2016), 测试集 (2017-2018)。
3. 将处理后的数据以 .npz 格式保存到 'data/processed' 文件夹中。
这个过程会消耗一些时间, 因为它需要读取所有已下载的数据, 请耐心等待...

数据源路径 (root_dir): E:\Project_Collection\2025\WEATHER\CLIMATE_LEARN_PROJECT\data
保存路径 (save_dir): E:\Project_Collection\2025\WEATHER\CLIMATE_LEARN_PROJECT\data\processed

100%|██████████| 36/36 [00:11<00:00, 3.02it/s]
100%|██████████| 2/2 [00:00<00:00, 4.27it/s]
100%|██████████| 1/1 [00:00<00:00, 4.73it/s]

=====
数据处理阶段成功结束!
请检查 'E:\Project_Collection\2025\WEATHER\CLIMATE_LEARN_PROJECT\data\processed' 文件夹, 确认 .npz 文件已生成。

进程已结束, 退出代码为 0
```

完整代码可见“2_process_data.py”

3.加载数据与建立基准 (Baseline)

在真正开始训练一个复杂的深度学习模型之前, 科学研究的标准流程是先建立一个“基准” (Baseline)。基准模型是非常简单的模型, 它们的作用是提供一个“最低标准”**。

如果复杂模型预测得还不如这些简单模型, 那就说明模型设计可能有问题。目标就是要**超越**这些基准。

在本实验中, 用到了两个基准

1. **气候学 (Climatology) 模型**: 这个模型非常“朴素”, 它的预测逻辑是: “明天的天气, 就是历史上所有‘明天’的平均值”。它会使用训练数据中所有年份的同一天、同一时刻的平均值作为预测结果。
2. **持续性 (Persistence) 模型**: 这个模型也很直接, 它的逻辑是: “24小时后的天气, 就和现在一样”。它直接用当前时刻的数据作为对未来的预测。这个方法对于短期预测 (比如1小时后) 通常效果不错。

```

# 这是项目的核心配置之一，它告诉模型我们想做什么
dm = cl.data.IterDataModule(
    task="direct-forecasting", # 任务类型：直接预测
    inp_root_dir=processed_data_dir, # 输入数据来自我们
处理好的文件夹
    out_root_dir=processed_data_dir, # 输出数据也来自那里（因为我们要预测未来的自己）
    in_vars=["temperature", "geopotential"], # 输入变量
    out_vars=["temperature", "geopotential"], # 输出变量

    src="era5",
    subsample=6, # 数据采样频率：每6小时一次
    pred_range=24, # 预测范围：预测未来24小时后的情况
    history=3, # 历史数据长度：用过去3个时间点的数据(t, t-6h, t-12h)来做预测
    batch_size=32 # 批处理大小：每次给模型32个样本
)

# --- 2. 加载并测试“气候学”基准模型 ---
climatology = cl.load_forecasting_module(
    data_module=dm,
    architecture="climatology" # 指定加载“气候学”模型
)

```

得到的测试结果如下所示

```
ClimateLearn快速入门 - 阶段3: 建立并测试基准模型
=====
检测到可用设备: cuda

正在设置数据加载器 (DataModule)...
将从这个路径加载数据: E:\Project_Collection\2025\WEATHER\CLIMATE_LEARN_PROJECT\data\processed
数据加载器设置完成。

--- 开始测试 Climatology (气候学) 模型 ---
Loading architecture: climatology
Using optimizer associated with architecture
Using learning rate scheduler associated with architecture
Loading training loss: lat_mse
No train transform
Loading validation loss: lat_rmse
Loading validation loss: lat_acc
Loading validation loss: lat_mse
Loading validation transform: denormalize
Loading validation transform: denormalize
No validation transform
Loading test loss: lat_rmse
Loading test loss: lat_acc
Loading test transform: denormalize
Loading test transform: denormalize
Using default 'ModelCheckpoint'. Consider installing 'litmodels' package to enable 'LitModelCheckpoint' for auto
GPU available: True (cuda), used: True
TPU available: False, using: 0 TPU cores
HPU available: False, using: 0 HPUs
You are using a CUDA device ('NVIDIA GeForce RTX 5080 Laptop GPU') that has Tensor Cores. To properly utilize t
LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [GPU-ec76e680-13dd-7213-855f-4c63bbc4a3ce]
D:\Anaconda\envs\rag_env\lib\site-packages\pytorch_lightning\trainer\connectors\data_connector.py:425: The 'tes
Testing DataLoader 0: |          | 43/? [00:00<00:00, 46.54it/s]
```

Test metric	DataLoader 0
test/lat_acc:aggregate	0.002286637867969616
test/lat_acc:geopotential	-0.0020452924620714425
test/lat_acc:temperature	0.006618568198010681
test/lat_rmse:aggregate	534.9848123508632
test/lat_rmse:geopotential	1064.5965083803287
test/lat_rmse:temperature	5.373116321397814

--- Climatology 模型测试完成 ---

--- 开始测试 Persistence (持续性) 模型 ---

Loading architecture: persistence

Using optimizer associated with architecture

Using learning rate scheduler associated with architecture

Loading training loss: lat_mse

No train transform

Loading validation loss: lat_rmse

Loading validation loss: lat_acc

Loading validation loss: lat_mse

Loading validation transform: denormalize

Loading validation transform: denormalize

No validation transform

Loading test loss: lat_rmse

Loading test loss: lat_acc

Loading test transform: denormalize

Loading test transform: denormalize

LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [GPU-ec76e680-13dd-7213-855f-4c63bbc4a3ce]

Testing DataLoader 0: | | 43/? [00:00<00:00, 59.22it/s]

Test metric	DataLoader 0
test/lat_acc:aggregate	0.8103965440378824
test/lat_acc:geopotential	0.8191593687462576
test/lat_acc:temperature	0.8016337193295074
test/lat_rmse:aggregate	303.2970449929333
test/lat_rmse:geopotential	603.4956555367604
test/lat_rmse:temperature	3.0984344491063727

--- Persistence 模型测试完成 ---

- 气候学 (Climatology) 模型分数:

- test/lat_rmse:aggregate : **534.98**
- 这代表, 如果猜未来24小时的天气, 永远都用“历史上的平均天气”, 那么平均误差是这个数。

- 持续性 (Persistence) 模型分数:

- `test/l\_rmse:aggregate`: ****303.30****

- 这代表, 如果猜未来24小时的天气, 就用“当前的天气状况”, 那么平均误差是这个数。

接下来，下一个目标，就是想办法，确保深度学习训练出来的模型的结果，要比以上两种传统的方法要好！

4. 训练深度学习模型ResNet

ResNet 是一种非常经典的深度学习模型，它通过一种巧妙的“跳跃连接”结构，使得网络可以建得非常深，从而学习到数据中更复杂、更深层次的规律，同时又不容易产生梯度消失或爆炸的问题。

构建这样一个模型包含以下几个步骤：

1. **定义并加载 ResNet 模型。**
2. **设置“教练” (Trainer)**，并为它配置一些高级功能，比如：
 - **提前终止 (EarlyStopping)**: 如果发现模型在验证集上的表现不再提升，就自动停止训练，防止过拟合，并节省时间。
 - **模型检查点 (ModelCheckpoint)**: 在整个训练过程中，只保存那个在验证集上表现最好的模型。
 - **日志记录器 (TensorBoardLogger)**: 将训练过程中的所有指标（如损失函数的变化）都记录下来，方便事后进行可视化分析。
3. **开始训练 (fit)**：用训练集和验证集来“教”和“考”模型。
4. **最终测试 (test)**：用“最终考卷”（测试集）来评估训练出的最佳模型的最终性能。

4.1 神经网络的概念

想象一个正在学习辨认苹果的婴儿。你不会给他一堆规则（“如果物体是红色、圆形、有柄……”），而是直接拿一个苹果给他看，告诉他：“这是苹果”。你看得越多，他就认得越准。

神经网络 就是一个模仿婴儿大脑学习方式的计算机程序。

- **神经元 (Neuron)**: 是大脑中最基本的信息处理单元。在程序里，它是一个小小的计算单元，接收一些输入数字，进行简单的计算，然后输出一个新的数字。

- **网络 (Network):** 我们把成千上万个这样的“神经元”组织成**层 (Layers)**。

- ****输入层 (Input Layer)**:** 像婴儿的“眼睛”，负责“看到”数据。在我们的项目中，输入层看到的的就是当前天气图（温度、位势高度）。
- ****隐藏层 (Hidden Layers)**:** 像婴儿的“大脑皮层”，负责处理信息。输入的数据会在这里经过一层又一层的神经元进行极其复杂的计算和传递。这是“思考”发生的地方。
- ****输出层 (Output Layer)**:** 像婴儿的“嘴巴”，负责给出答案。在我们的项目中，它输出的就是对**24**小时后天气图的预测。

学习过程 (Training):

我们给网络看一个“问题”（当前天气）和“标准答案”（24小时后的真实天气）。网络会先猜一个“答案”。我们计算它的猜测和“标准答案”之间的**差距（我们称之为“误差”或“损失 (Loss)”**）。然后，我们用这个“误差”反过来微调网络中所有神经元的连接方式，让它下一次猜得更准一点。这个微调的过程叫做**反向传播 (Backpropagation)**。把这个过程重复几百万次，网络就会变得越来越“聪明”。

4.2 深度学习的概念

当一个神经网络包含了很多很多个隐藏层时（比如几十上百层），就称之为**深度神经网络 (Deep Neural Network)**，使用这种网络的过程就叫做**深度学习 (Deep Learning)**。

- **层数越深，能力越强:** 浅层的网络可能只能识别一些简单的特征（比如“这个区域温度高”）。而深层网络能组合这些简单特征，理解更复杂的、抽象的概念（比如“一个正在向东南方向移动的冷锋系统”）。

4.3 ResNet (残差网络)

理论上，网络越深越好。但实际中，当网络堆得太深时，会出现一个问题：**梯度消失 (Vanishing Gradients)**。

想象在玩一个传话游戏，信息经过太多人传递后，到最后就完全失真了。在深度网络里，“误差”这个信号在从后往前传导进行微调时，经过太多层也会变得极其微弱，导致前面的层几乎学不到东西。

ResNet (残差网络) 是一种革命性的架构，它巧妙地解决了这个问题。

- **核心思想：跳跃连接 (Skip Connection)**

ResNet 允许信息“抄近道”。除了常规的一层层传递，它还会把某一层输入的信息，直接“跳”过几层，加到后面几层的输出上。

- **类比：**

想象一位专家（一层网络）在处理一项任务。他的任务不是从零开始创造一个全新的解决方案，而是基于前人（前一层）的工作，只做出一些**修正和改进**。这个“修正和改进”的部分，就是所谓的“**残差**” (Residual)。

这个“跳跃连接”保证了原始信息不会在深层传递中丢失，使得梯度（误差信号）可以畅通无阻地传播回去，让整个非常深的网络都能有效训练。 **

4.4 具体的工作

4.4.1 阶段一：准备工作 (Setup)

1. `import ...` :

导入所有需要的工具箱，比如 `climate_learn (cl)`、`pytorch_lightning (pl)` 等。

2. `torch.set_float32_matmul_precision('high')` :

利用 RTX 5080 显卡的 Tensor Cores，使用一种混合精度模式进行计算，**大大加速训练**。

4.4.2 阶段二：数据后勤 (IterDataModule)

1. `dm = cl.data.IterDataModule(...)`:

创建一个“数据后勤总管”。它的职责是：

- **定位数据**: 找到 `data/processed` 文件夹里处理好的数据。
- **定义任务**: 告诉它任务是 `direct-forecasting` (直接预测)。
- **定义输入输出**: 告诉它用 `temperature` 和 `geopotential` 来预测未来的 `temperature` 和 `geopotential`。
- **设定参数**: 我们告诉它，用过去3个时间点 (`history=3`) 的数据，来预测未来24小时 (`pred_range=24`) 的情况。
- **批处理** (`batch_size=128`): 为了效率，GPU不会一张一张地处理天气图，而是一次处理128个样本 (一批)。
- **工人** (`num_workers=4`): 你指定了8个CPU核心作为“工人”，在后台不停地准备下一批数据，这样GPU就不用停下来等待，大大提升了效率。

4.4.3 阶段三：召唤AI大脑 (`load_forecasting_module`)

1. `model = cl.load_forecasting_module(...)`:

这一行，正在实例化 **ResNet 模型**。 `architecture="rasp-theurey-2020"` 就是一个 `climate-learn` 库里预先定义好的、被证明在气象预测上效果不错的 ResNet 配置。这个 `model` 对象，就是“AI大脑”。

4.4.4 阶段四：聘请教练团队 (`Callbacks`)

训练一个模型不能放任不管，一个专业的“教练团队”来监督整个过程。

1. `RichProgressBar()`:

一个“现场评论员”，用漂亮的进度条实时播报训练进展。

2. `EarlyStopping(...)`:

一位“理智的教练”。它会盯着模型的验证集表现

(val/lat_mse:aggregate)。如果发现连续5个周期 (epoch) 模型的表现都没有进步，它就会提前叫停训练。这能有效**防止模型过度学习（过拟合）并节省大量时间**。

3. ModelCheckpoint(...):

一位“档案管理员”。它同样盯着验证集表现，但它的工作是**只保存到目前为止表现最好的那个模型**。训练结束后，我们拿来测试的，就是这个“历史最佳模型”，而不是最后训练完的那个。

4.4.5 阶段五：任命总教练并开始训练 (Trainer.fit)

1. trainer = pl.Trainer(...):

创建“总教练”。我们把所有东西都交给它：

- `accelerator="gpu"`：命令它使用GPU。
- `callbacks=[...]`：把刚才聘请的教练团队交给它。
- `max_epochs=20`：告诉它最多训练20个完整的学习周期。
- `precision="16-mixed"`：指示它使用我们之前设定的高速混合精度模式。

2. trainer.fit(model, datamodule=dm):

这是按下“开始”按钮的时刻！ 总教练 `trainer` 开始了核心的训练循环：

- 从后勤总管 `dm` 那里拿一批数据。
- 把数据喂给AI大脑 `model`。
- 计算出“误差”。
- 根据误差，微调 `model` 的参数。
- 周而复始，同时教练团队在一旁监督、记录、存档。

4.4.6 阶段六：最终大考 (Trainer.test)

1. trainer.test(datamodule=dm, ckpt_path="best"):

训练结束后（无论是达到20个周期还是被提前叫停），这一行命令启动“最终大考”。

- 总教练会自动加载档案管理员 ``ModelCheckpoint`` 保存下来的那个****历史最佳模型**** (``ckpt_path="best"``)。
- 它会从后勤总管 ``dm`` 那里拿到****测试集****——这是一份模型在训练和验证阶段从未见过的、全新的“考卷”。
- 它计算出模型在这份终极考卷上的分数，并把结果打印出来。

如下图所示，为训练过程！

```
即将开始模型训练... 这将花费较长时间, 请耐心等待。
目标: 挑战 Persistence 模型的 'lat_rmse' 分数 (303.30)。
监控指标: val/lat_mse:aggregate (越低越好)
=====
2025-06-15 20:01:03.386541: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2025-06-15 20:01:05.402156: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
D:\Anaconda\envs\AI-Foundation\Lib\site-packages\pytorch_lightning\callbacks\model_checkpoint.py:654: Checkpoint directory E:\Project_Collection\2025\WEATHER\CLIMATE_LEARN_PROJECT\resnet_forecasting_24hrs\checkpoints exists and is not empty
LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [GPU-ec76e680-13dd-7213-855f-4c63bbc4a3ce]



|   | Name | Type   | Params | Mode  |
|---|------|--------|--------|-------|
| 0 | net  | ResNet | 5.7 M  | train |



Trainable params: 5.7 M
Non-trainable params: 0
Total params: 5.7 M
Total estimated model params size (MB): 22
Modules in train mode: 238
Modules in eval mode: 0
D:\Anaconda\envs\AI-Foundation\Lib\site-packages\pytorch_lightning\trainer\connectors\data_connector.py:420: Consider setting 'persistent_workers=True' in 'val_dataloader' to speed up the dataloader worker initialization.
D:\Anaconda\envs\AI-Foundation\Lib\site-packages\pytorch_lightning\trainer\connectors\data_connector.py:420: Consider setting 'persistent_workers=True' in 'train_dataloader' to speed up the dataloader worker initialization.
Epoch 1/19 166 / -- 0:02:05 * --:--:-- 2.14it/s v_num: 3.000 train/lat_mse:aggregate: 0.025

D:\Anaconda\envs\AI-Foundation\Lib\site-packages\pytorch_lightning\trainer\connectors\data_connector.py:420: Consider setting 'persistent_workers=True' in 'val_dataloader' to speed up the dataloader worker initialization.
D:\Anaconda\envs\AI-Foundation\Lib\site-packages\pytorch_lightning\trainer\connectors\data_connector.py:420: Consider setting 'persistent_workers=True' in 'train_dataloader' to speed up the dataloader worker initialization.
Epoch 1/19 384 / -- 0:03:57 * --:--:-- 2.13it/s v_num: 3.000 train/lat_mse:aggregate: 0.023

Validation 18 / -- 0:00:02 * --:--:-- 6.44it/s
```

4.5 训练结果

训练结束后，可发现最终的结果如下图所示

```
D:\Anaconda\envs\AI-Foundation\Lib\site-packages\pytorch_lightning\trainer\connector
setting 'persistent_workers=True' in 'train_dataloader' to speed up the dataloader w
Epoch 19/19 384/-- 0:03:56 • -:--:-- 2.14it
Epoch 19/19 384/-- 0:03:57 • -:--:-- 2.14it

=====
模型训练结束！
=====

开始使用测试集评估已保存的最佳模型...
Restoring states from the checkpoint path at E:\Project_Collection\2025\WEATHER\CLIM
LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [GPU-ec76e680-13dd-7213-855f-4c63bbc4a3ce]
Loaded model weights from the checkpoint at E:\Project_Collection\2025\WEATHER\CLIM
D:\Anaconda\envs\AI-Foundation\Lib\site-packages\pytorch_lightning\trainer\connector
tialization.
```

Test metric	DataLoader 0
test/lat_acc:aggregate	0.9503129673009765
test/lat_acc:geopotential	0.9684467973286343
test/lat_acc:temperature	0.9321791372733184
test/lat_rmse:aggregate	127.66343289703494
test/lat_rmse:geopotential	253.52085512285734
test/lat_rmse:temperature	1.806010671212567

```
Testing 16/-- 0:00:02 • -:--:-- 6.00it/s

=====
阶段4成功结束！
请查看上方表格中的 'test/lat_rmse:aggregate' 分数，并与基准模型对比。
```

可以发现，此时训练的ResNet模型取得了127.66 的结果，比
气候学 (Climatology) 模型分数:534.98
持续性 (Persistence) 模型分数： 303.30
都要好。

从其他指标上

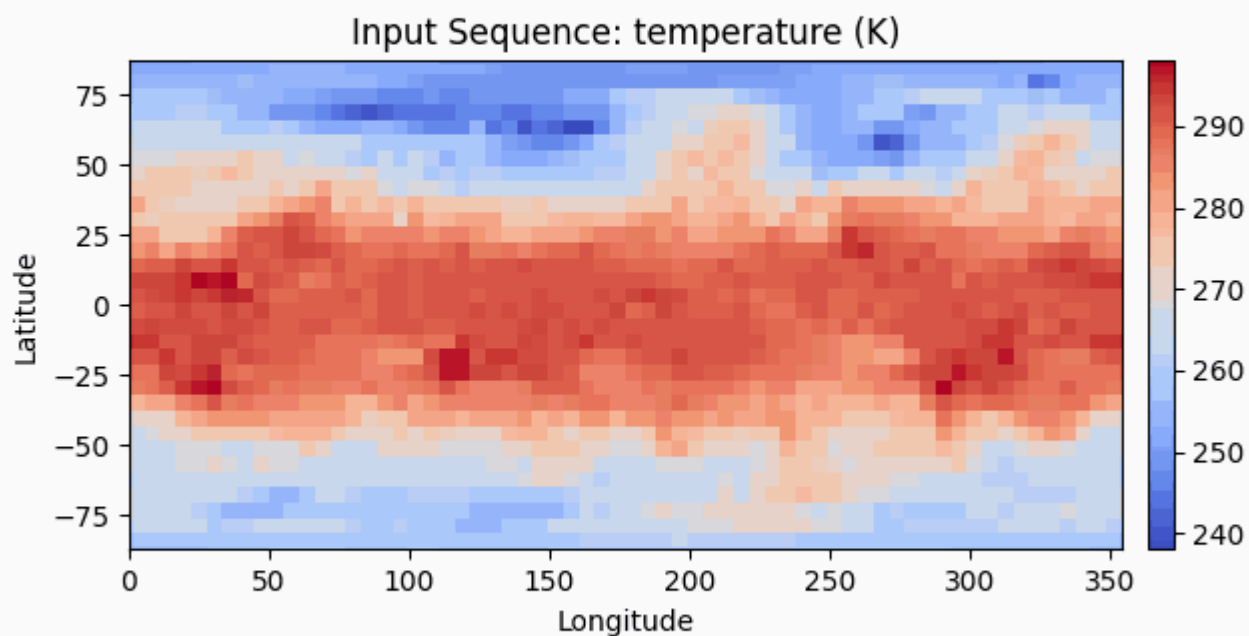
- test/lat_acc:aggregate : 0.95，这代表我们模型的预测结果与真实结果之间的相关性高达95%，这是一个非常高的准确度。
 - test/lat_rmse:geopotential 和 test/lat_rmse:temperature : 这两项是两个天气变量各自的误差。可以看到，模型在预测温度（误差仅1.8）方面做得极其出色。
- 具体的代码可见”4_train_resnet_model.py“

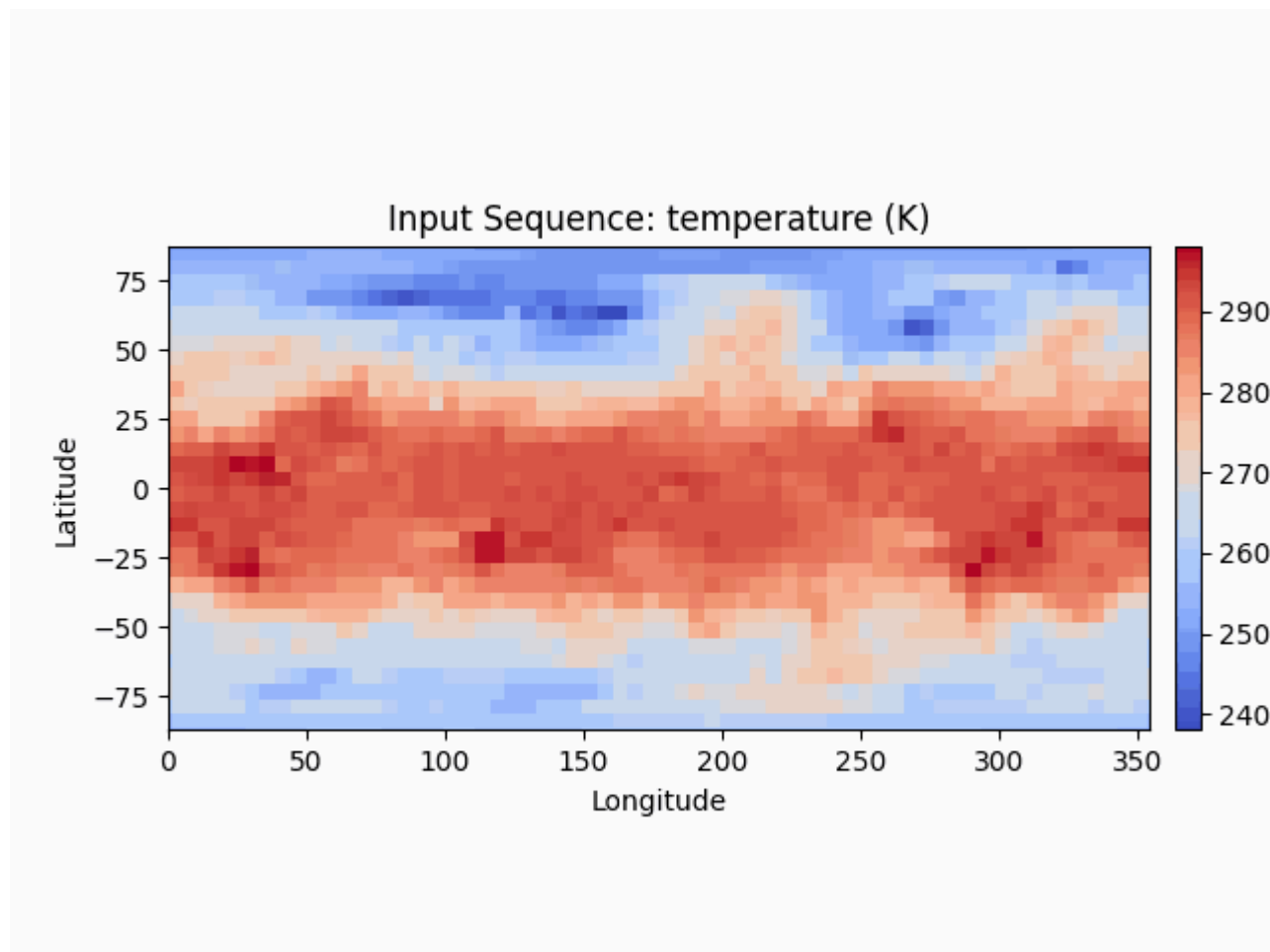
5.结果可视化

调用visualize_at_index 函数绘图并保存为 GIF。

5.1 温度对比

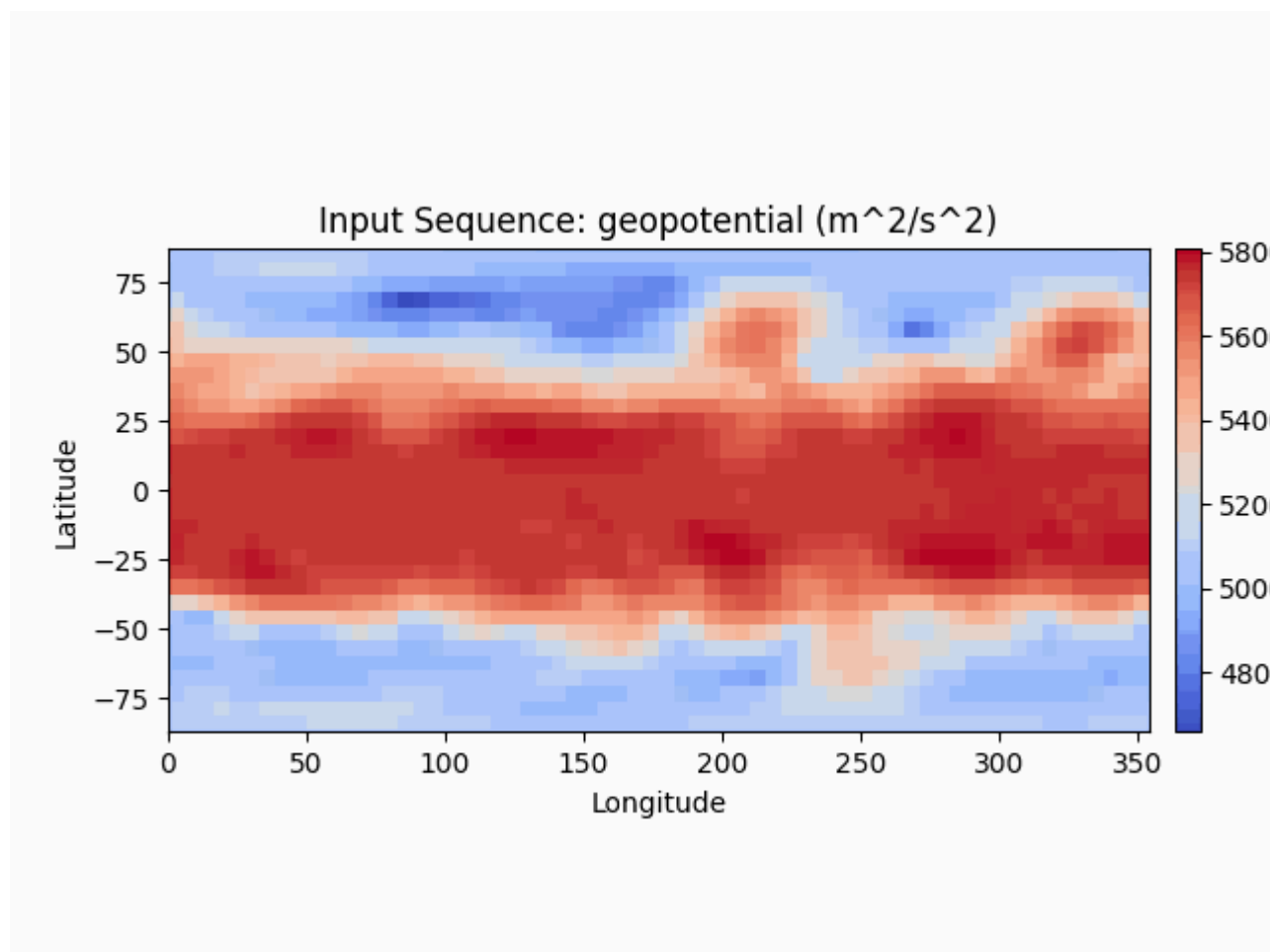
ResNet AI 模型预测



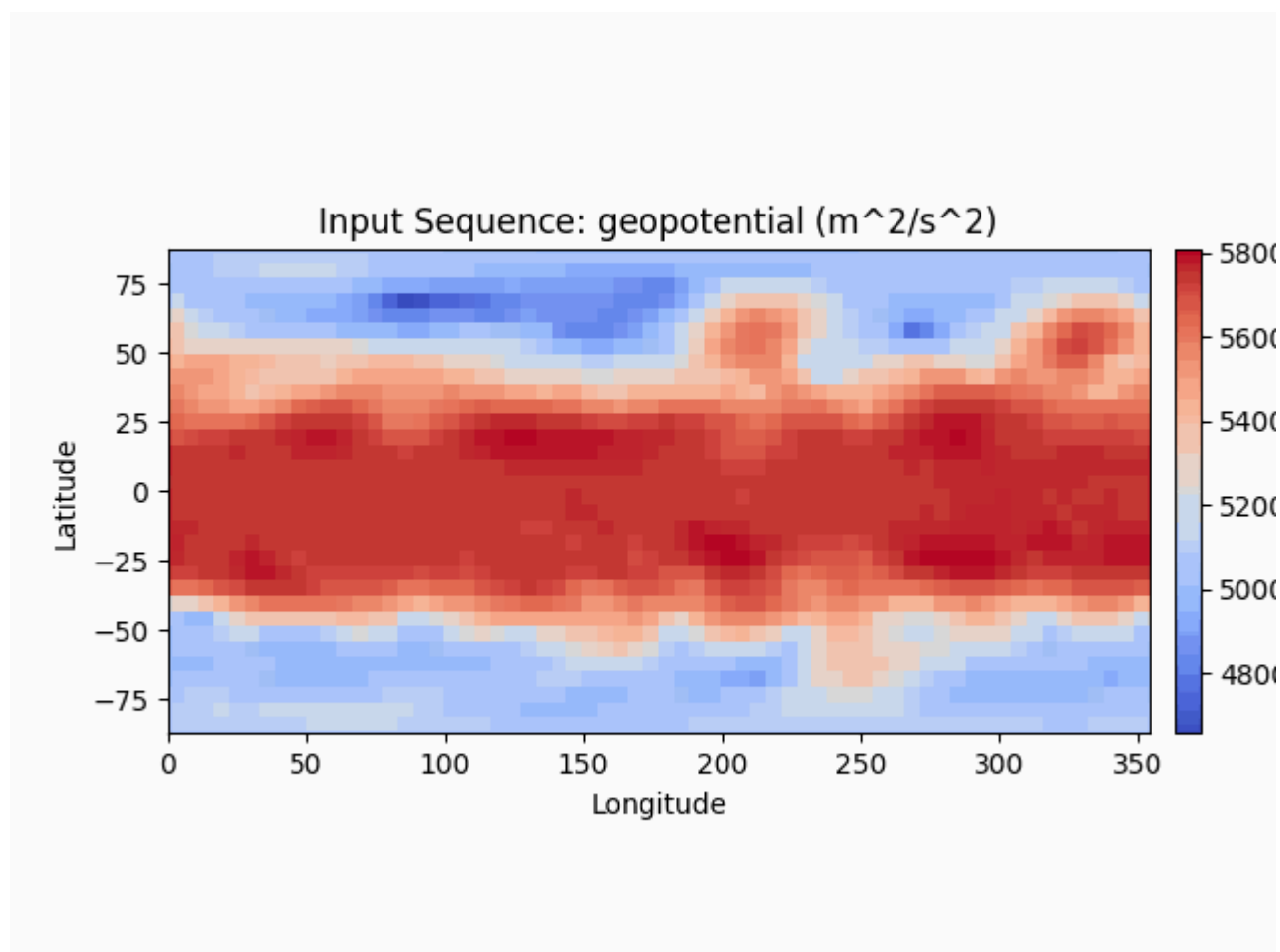


5.2 位势高度对比

ResNet AI 模型预测



Persistence 基准模型预测



5.3 网站构建

同时，为了更好的进行展示，通过Streamlit进行了结果的展示。

